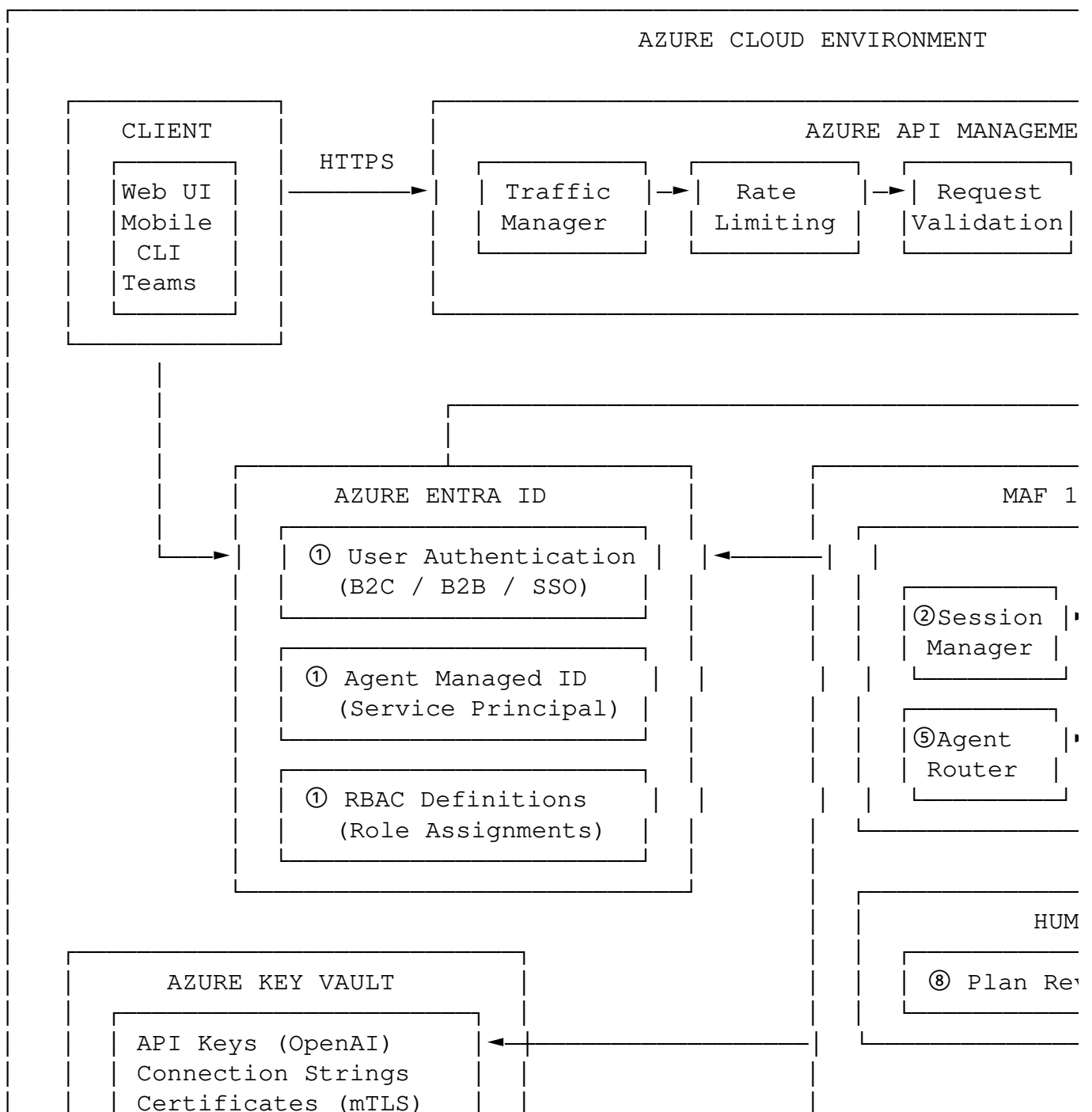


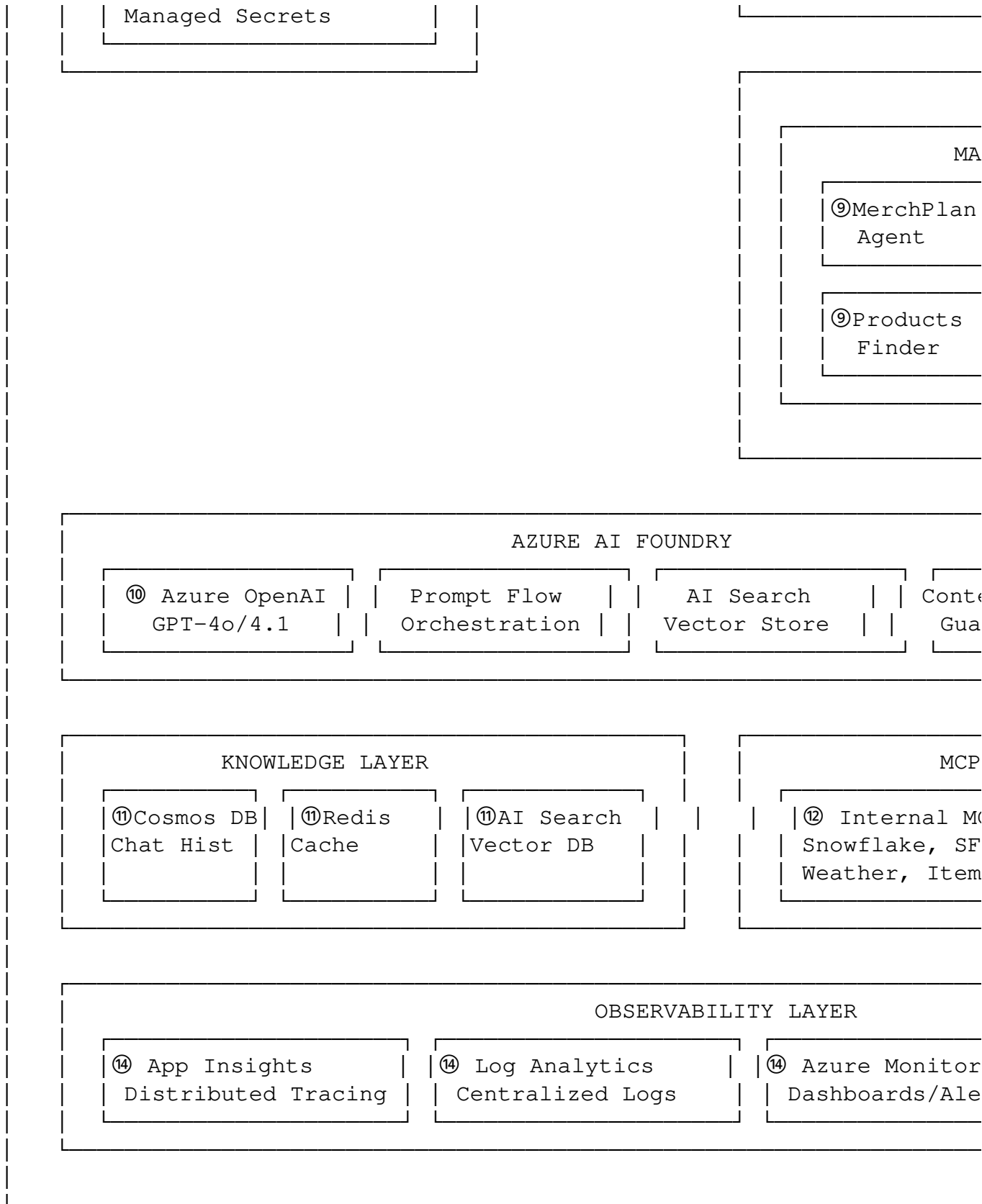
Architecture Version 1: Azure-Native (Full Azure Ecosystem)

Overview

This architecture leverages the complete Azure ecosystem for enterprise-grade deployment, utilizing **Azure AI Foundry**, **Azure API Management (APIM)**, **Azure Entra ID**, **Azure Key Vault**, and the **MAF 1.0 GA Framework**.

Architecture Diagram (ASCII)





Mermaid Diagram

```
graph TD
    subgraph Client ["👤 Step 1: User Initiates Request"]
        User["User (Web/Mobile/Teams)"]
    end
```

```

subgraph APIM["🌐 Step 2-3: API Gateway Processing"]
    Traffic["2a. Traffic Management"]
    RateLimit["2b. Rate Limiting Check"]
    Validation["2c. Request Validation"]
    OAuth["3. OAuth2 Token Extraction"]
end

subgraph EntraID["🔑 Step 4: Authentication & Authorization"]
    TokenValidation["4a. JWT Token Validation"]
    ClaimsExtract["4b. Claims Extraction"]
    RBACCheck["4c. RBAC Permission Check"]
end

subgraph KeyVault["🔑 Secrets Management"]
    Secrets["API Keys & Certificates"]
end

subgraph MAF_Orch["⚙️ Step 5-8: MAF Orchestration"]
    Session["5. Session Manager<br/>(Create/Resume Session)"]
    Planner["6. LLM Planner<br/>(Generate Task-Ledger)"]
    Context["7. Context Builder<br/>(Memory + Tools)"]
    Router["8. Agent Router<br/>(Select Agent)"]
end

subgraph HITL["👁️ Step 9: Human Approval (Optional)"]
    Approval["Plan Review & Approval"]
end

subgraph Agents[" Step 10: Agent Execution"]
    Agent1["MerchPlanner"]
    Agent2["SpacePlanner"]
    Agent3["LoyaltyAgent"]
    Agent4["ProductsFinder"]
    Agent5["CommercialSales"]
    Agent6["CampaignAnalyst"]
end

subgraph Foundry[" Step 11: LLM Inference"]
    OpenAI["Azure OpenAI<br/>(GPT-4o)"]
    Safety["Content Safety"]
end

subgraph MCP["🔗 Step 12: Tool Execution"]
    Tools["MCP Servers<br/>(Snowflake, Salesforce, etc.)"]
end

subgraph Synth["📝 Step 13: Response"]
    Synthesizer["Result Synthesizer"]
    Response["Final Response"]
end

User -->|"1. HTTPS Request"| Traffic

```

```
Traffic --> RateLimit --> Validation --> OAuth
OAuth -->|"JWT Token"| TokenValidation
TokenValidation --> ClaimsExtract --> RBACCheck
RBACCheck -->|"Authorized"| Session
KeyVault -.->|"Secrets"| MAF_Orch
Session --> Planner --> Context --> Router
Router -->|"Complex Task"| Approval
Approval -->|"Approved"| Agents
Router -->|"Simple Task"| Agents
Agents --> OpenAI
OpenAI --> Safety
Agents --> Tools
Tools --> Agents
Agents --> Synthesizer
Synthesizer --> Response
Response -->|"14. Return to User"| User
```

Step-by-Step Flow Narrative

Step 1: User Initiates Request

Actor: End User (via Web UI, Mobile App, Teams Bot, or CLI)

What Happens:

1. User types a query like "Analyze paint category sell-through rate for Q4"
2. Client application packages the request as JSON:

```
{
  "query": "Analyze paint category sell-through rate for Q4",
  "conversation_id": "conv-abc123"
}
```

3. Request is sent via HTTPS to the Azure API Management endpoint

Example:

```
POST https://mafga-api.azure-api.net/v1/orchestration/run
Authorization: Bearer eyJhbGciOiJSUzI1NiIs...
Content-Type: application/json
```

```
{
  "goal": "Analyze paint category sell-through rate for Q4",
  "orchestration_type": "magentic",
  "require_approval": false
}
```

Step 2: API Management - Traffic Processing

Component: Azure API Management (APIM)

What Happens:

1. **2a. Traffic Manager** receives the inbound request
 - Routes to correct backend based on URL path
 - Load balances across multiple instances if configured
 2. **2b. Rate Limiting** checks quota
 - Per-user rate limits (e.g., 100 requests/minute)
 - Per-subscription quotas (e.g., 10,000 requests/day)
 - Returns 429 Too Many Requests if exceeded
 3. **2c. Request Validation** ensures schema compliance
 - Validates JSON structure against OpenAPI schema
 - Checks required fields are present
 - Returns 400 Bad Request if invalid
-

Step 3: OAuth2 Token Extraction

Component: Azure API Management (APIM)

What Happens:

1. APIM extracts the `Authorization: Bearer <token>` header
2. Validates token format (must be valid JWT)
3. Passes token to Entra ID for validation

APIM Policy (Inbound):

```
<inbound>
  <validate-jwt header-name="Authorization"
    failed-validation-httpcode="401"
    failed-validation-error-message="Unauthorized">
    <openid-config url="https://login.microsoftonline.com/{tenant-id}/oauth2-1.0/openid-configuration"
      <audiences>
        <audience>api://mafga-multiagent</audience>
      </audiences>
      <required-claims>
        <claim name="roles" match="any">
          <value>Agent.Invoke</value>
        </claim>
      </required-claims>
    </validate-jwt>
  </inbound>
```

Step 4: Authentication & Authorization

Component: Azure Entra ID

What Happens:

1. 4a. JWT Token Validation

- Verifies token signature using Entra ID public keys
- Checks token expiration (`exp` claim)
- Validates issuer (`iss` claim matches tenant)
- Validates audience (`aud` claim matches app registration)

2. 4b. Claims Extraction

- Extracts user identity (`sub`, `oid`, `upn` claims)
- Extracts roles (`roles` claim)
- Extracts groups (`groups` claim)

3. 4c. RBAC Permission Check

- Matches user roles against required permissions
- Agent-level access control:

User Role: "Retail.Analyst" → Can access: MerchPlanner, SpacePl
User Role: "Sales.Manager" → Can access: CommercialSales, Campa
User Role: "Admin" → Can access: All agents

UserContext Created:

```
@dataclass
class UserContext:
    user_id: str = "user-12345"
    email: str = "analyst@company.com"
    name: str = "John Doe"
    roles: list[str] = ["Retail.Analyst", "Agent.Invoke"]
    groups: list[str] = ["Retail-Team", "AU-Region"]
```

Step 5: Session Manager

Component: MAF 1.0 GA Orchestration Layer

What Happens:

1. Checks if `conversation_id` exists in request
2. If exists: Loads existing session from Azure Cosmos DB
3. If not: Creates new session with unique ID
4. Initializes session state:

```
session = {
    "session_id": "sess-xyz789",
    "conversation_id": "conv-abc123",
    "user_context": user_context,
    "created_at": "2026-04-15T10:30:00Z",
    "messages": [],
    "plan": None,
    "status": "active"
}
```

Step 6: LLM Planner (Task-Ledger Creation)

Component: Magentic Orchestrator

What Happens:

1. Retrieves conversation history from Cosmos DB
2. Loads relevant context from Azure AI Search (RAG)
3. Calls Azure OpenAI with planning prompt:

Prompt Sent to GPT-4o:

You are a planning agent. Create an execution plan for this goal:
"Analyze paint category sell-through rate for Q4"

Available agents:

- MerchPlanner: Merchandise planning, inventory analysis, sales forecasts
- SpacePlanner: Store layout, planogram optimization
- CampaignAnalyst: Marketing campaigns, weather impact analysis

Output JSON plan with steps, agent assignments, and dependencies.

LLM Response:

```
{
  "plan_id": "plan-001",
  "goal": "Analyze paint category sell-through rate for Q4",
  "steps": [
    {
      "step_id": "step_1",
      "agent_name": "MerchPlanner",
      "task": "Query Q4 paint category sales and inventory data",
      "dependencies": []
    },
    {
      "step_id": "step_2",
      "agent_name": "CampaignAnalyst",
      "task": "Analyze promotional impact on paint sales",
      "dependencies": ["step_1"]
    }
  ]
}
```

Step 7: Context Builder

Component: Magentic Orchestrator

What Happens:

1. **Load Memory:** Retrieves relevant past conversations from Redis cache
2. **Load Skills:** Reads SKILL.md files for selected agents
3. **Load Tools:** Binds MCP tools to each agent
4. **Build Context Window:**

```
context = {
    "conversation_history": [...last 10 messages...],
    "retrieved_documents": [...RAG results from AI Search...],
    "agent_skills": "# Retail Analysis Skill\n...",
    "available_tools": ["query_sales", "get_inventory", "get_weathe
}
```

Step 8: Agent Router

Component: Magentic Orchestrator

What Happens:

1. Reads the plan's first ready step
2. Selects appropriate agent based on agent_name
3. Routes task to selected agent
4. For parallel execution: Uses `asyncio.gather()` to run independent steps concurrently

Routing Logic:

```
# Get steps with no pending dependencies
ready_steps = plan.get_ready_steps()

# Route to agents
for step in ready_steps:
    agent = agent_registry.get(step.agent_name)
    task_queue.add(agent.invoke(step.task, context))
```

Step 9: Human-in-the-Loop (HITL) Approval

Component: Human Approval Manager (Optional)

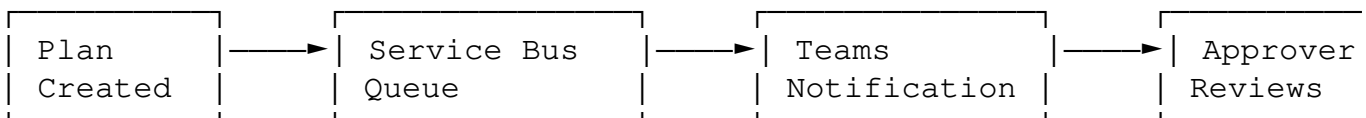
When Triggered:

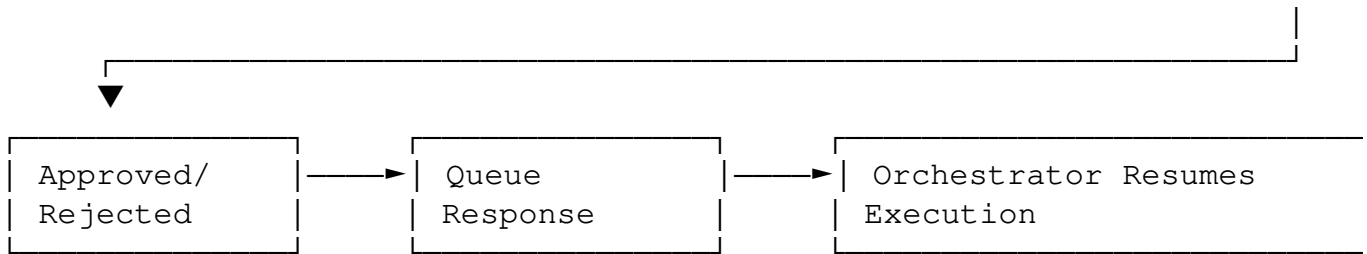
- `require_approval: true` in request
- Plan involves sensitive operations (e.g., price changes)
- Plan cost exceeds threshold

What Happens:

1. Plan is serialized and stored in Azure Service Bus queue
2. Notification sent to approver (Teams/Email)
3. System waits for approval (with timeout)
4. Approver reviews plan and approves/rejects

Approval Workflow:





Step 10: Agent Execution

Component: MAF Agent Layer

What Happens:

1. Selected agent (e.g., MerchPlanner) receives task
2. Agent constructs prompt with system instructions + task
3. Agent invokes tools as needed via MCP
4. Agent processes LLM response

Example Agent Execution:

```
# MerchPlanner Agent
class MerchPlannerAgent(BaseRetailAgent):
    async def invoke(self, task: str, context: dict) -> AgentResponse:
        # 1. Build prompt
        messages = [
            {"role": "system", "content": self.instructions},
            {"role": "user", "content": task}
        ]

        # 2. Call LLM
        response = await self.chat_client.chat_completion(messages)

        # 3. If LLM wants to use tool
        if response.tool_calls:
            for tool_call in response.tool_calls:
                # Call MCP server
                tool_result = await self.mcp_client.call_tool(
                    server="snowflake_mcp",
                    tool="query_sales",
                    arguments={"category": "paint", "period": "Q4"}
                )
                # Continue conversation with tool result
                messages.append({"role": "tool", "content": tool_result})

        # 4. Get final response
        final_response = await self.chat_client.chat_completion(messages)
        return AgentResponse(content=final_response.content)
```

Step 11: LLM Inference (Azure AI Foundry)

Component: Azure OpenAI (via Azure AI Foundry)

What Happens:

1. Request sent to Azure OpenAI endpoint
2. Content Safety filters check input
3. Model generates response
4. Content Safety filters check output
5. Response returned with usage metrics

Configuration:

```
# Azure OpenAI Configuration (from Key Vault)
client = AzureOpenAI(
    azure_endpoint="https://foundry-project.openai.azure.com",
    api_key=key_vault.get_secret("azure-openai-key"),
    api_version="2024-02-01"
)

response = await client.chat.completions.create(
    model="gpt-4o",
    messages=messages,
    tools=tool_definitions,
    temperature=0.3
)
```

Step 12: MCP Tool Execution

Component: MCP Servers (Snowflake, Salesforce, etc.)

What Happens:

1. Agent requests tool execution via MCP protocol
2. MCP server receives `tools/call` request
3. Server executes query against data source
4. Results returned to agent

MCP Call Example:

```
// Request
{
  "method": "tools/call",
  "params": {
    "name": "query_sales",
    "arguments": {
      "query": "SELECT category, SUM(sales) FROM sales_data WHERE categ
    }
  }
}

// Response
{
  "result": {
```

```
    "content": [
      {"type": "text", "text": "Paint category Q4 sales: $2.4M, Units: 45,230"}
    ]
  }
}
```

Step 13: Result Synthesis

Component: Result Synthesizer

What Happens:

1. Collects results from all executed agent steps
2. Calls LLM to synthesize into coherent response
3. Formats final response for user

Synthesis Prompt:

You are a synthesis agent. Combine these agent results into a coherent response.

Original Goal: "Analyze paint category sell-through rate for Q4"

Agent Results:

1. [MerchPlanner]: "Paint category Q4 sales: \$2.4M, 45,230 units, 72% sell-through"
2. [CampaignAnalyst]: "Holiday promotions drove 15% increase in paint sales"

Synthesize into a comprehensive response addressing the original goal.

Step 14: Response to User

Component: API Gateway → Client

What Happens:

1. Final response packaged as JSON
2. Sent through APIM (outbound policies apply)
3. Response logged to Application Insights
4. Delivered to client application

Final Response:

```
{
  "status": "completed",
  "result": "**Paint Category Q4 Analysis**\n\nThe paint category achieved a 72% sell-through rate, with sales of $2.4M and 45,230 units. Holiday promotions drove a 15% increase in sales."
  "plan": {
    "plan_id": "plan-001",
    "steps": [...]
  },
  "conversation_id": "conv-abc123",
  "metadata": {
    "tokens_used": 1847,
    "latency_ms": 3421,
  }
}
```

```
    "agents_invoked": ["MerchPlanner", "CampaignAnalyst"]
  }
}
```

Authentication Flow Detail

```
sequenceDiagram
    participant User
    participant Client as Client App
    participant APIM as Azure APIM
    participant Entra as Azure Entra ID
    participant MAF as MAF Service
    participant KV as Key Vault

    User->>Client: 1. Login Request
    Client->>Entra: 2. Redirect to Entra Login
    Entra->>User: 3. Login Page
    User->>Entra: 4. Credentials
    Entra->>Entra: 5. Validate & Generate Tokens
    Entra->>Client: 6. Access Token + Refresh Token

    Note over Client,APIM: User makes API request

    Client->>APIM: 7. API Request + Bearer Token
    APIM->>APIM: 8. Rate Limit Check
    APIM->>Entra: 9. Validate JWT Token
    Entra->>APIM: 10. Token Valid + Claims
    APIM->>MAF: 11. Forward Request + User Context
    MAF->>KV: 12. Get Secrets (API Keys)
    KV->>MAF: 13. Return Secrets
    MAF->>MAF: 14. Process Request
    MAF->>APIM: 15. Response
    APIM->>Client: 16. Response
    Client->>User: 17. Display Result
```

Security Considerations

Layer	Security Control	Implementation
Network	TLS 1.3	APIM enforces HTTPS
API Gateway	Rate Limiting	APIM policies
Authentication	OAuth2 / OIDC	Azure Entra ID
Authorization	RBAC	Entra ID roles + Custom claims
Secrets	Managed Access	Azure Key Vault + Managed Identity
Data	Encryption at rest	Azure Storage encryption
LLM	Content Safety	Azure AI Content Safety
Audit	Logging	Application Insights + Log Analytics

Key Benefits of Azure-Native Architecture

1. **Enterprise Security:** Full Entra ID integration with MFA, Conditional Access
2. **Compliance:** Azure compliance certifications (SOC 2, ISO 27001, HIPAA)
3. **Managed Services:** Reduced operational overhead
4. **Scalability:** Auto-scaling with APIM and Azure Functions
5. **Observability:** Unified monitoring with Azure Monitor
6. **Cost Management:** Azure Cost Management integration